Limited Time Offer: **Up to 25% off select mock interviews** 🎉

Patterns

Dealing with Contention

Learn about how to deal with high contention in your system design interview.

Dealing with Contention

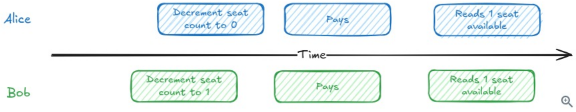
🔒 **Contention** occurs when multiple processes compete for the same resource simultaneously. This could be booking the last concert ticket, bidding on an auction item, or any similar scenario. Without proper handling, you get race conditions, double-bookings, and inconsistent state. This pattern walks you through solutions from simple database transactions to more complex distributed coordination, showing when optimistic concurrency beats pessimistic locking and how to scale beyond single-node constraints.

The Problem

Consider buying concert tickets online. There's 1 seat left for The Weeknd concert. Alice and Bob both want this last seat and click "Buy Now" at exactly the same moment. Without proper coordination, here's what happens:

1. Alice's request reads: "1 seat available"
2. Bob's request reads: "1 seat available" (both reads happen before either write)
3. Alice's request checks if 1 ≥ 1 (yes, there is a seat available), proceeds to payment
4. Bob's request checks if 1 ≥ 1 (yes, there is a seat available), proceeds to payment
5. Alice gets charged $500, seat count decremented to 0
6. Bob gets charged $500, seat count decremented to -1

Both Alice and Bob receive confirmation emails with the exact same seat number. They both show up to the concert thinking they own Row 5, Seat 12. One of them is getting kicked out, and the venue has to issue a refund while dealing with two very angry customers.



Race Condition Timeline

The race condition happens because both Alice and Bob read the same initial state (1 seat available) before either of their updates takes effect. By the time Bob's update runs, Alice has already reduced the count to 0, but Bob's logic was based on the stale reading of 1 seat.

This race condition occurs because reading and writing aren't atomic. There's a gap between "check the current state" and "update based on that state" where the world can change. In that tiny window (microseconds in memory, milliseconds over a network) lies chaos.

The problem only gets worse when you scale. With 10,000 concurrent users hitting the same resource, even small race condition windows create massive conflicts. As you continue to grow, it's likely that you'll need to coordinate across multiple nodes which adds to the complexity.

To get this right, we need some form of synchronization.

The Solution

The solution to contention problems follows a natural progression of complexity. We start with single-database solutions using atomicity and transactions, then add coordination mechanisms when concurrent access creates conflicts, and finally move to distributed coordination when multiple databases are involved.

Single Node Solutions

When all your data exists in a single database, contention solutions are more straightforward but still have important gotchas to watch out for. Let's walk through the possible solutions for handling contention within a single node.

Atomicity

Before reaching for complex coordination mechanisms, atomicity solves many contention problems. **Atomicity** means that a group of operations either all succeed or all fail. There's no partial completion. If you're transferring money between accounts, either both the debit and credit happen, or neither does.

Transactions are how databases provide atomicity. A transaction is a group of database operations treated as a single unit. You start with `BEGIN TRANSACTION`, perform your operations, and finish with `COMMIT` (to save changes) or `ROLLBACK` (to undo everything).

```
BEGIN TRANSACTION;

-- Debit Alice's account
UPDATE accounts SET balance = balance - 100 WHERE user_id = 'alice';

-- Credit Bob's account
UPDATE accounts SET balance = balance + 100 WHERE user_id = 'bob';

COMMIT; -- Both operations succeed together
```

If anything goes wrong during this transaction, like Alice has insufficient funds, Bob's account doesn't exist, or the database crashes, the entire transaction gets rolled back. This prevents money from disappearing or appearing out of nowhere.

> These examples use SQL because relational databases are well-known for their strong ACID guarantees (where the "A" in ACID stands for Atomicity). However, many databases support transactions, including some NoSQL databases like MongoDB (multi-document transactions), DynamoDB (transaction operations), and CockroachDB (distributed ACID transactions). The concepts apply regardless of the specific database technology.

For a concert ticket purchase, atomicity ensures that multiple related operations happen together. A ticket purchase isn't just decrementing a seat count - you also need to create a ticket record:

```
BEGIN TRANSACTION;

-- Check and reserve the seat
UPDATE concerts
SET available_seats = available_seats - 1
WHERE concert_id = 'weeknd_tour'

-- Create the ticket record
INSERT INTO tickets (user_id, concert_id, seat_number, purchase_time)
VALUES ('user123', 'weeknd_tour', 'A15', NOW());

COMMIT;
```

If any of these operations fail, the entire transaction rolls back. You don't end up with a seat reserved but no ticket created.

But even with this atomic transaction, there's a subtle problem that atomicity alone doesn't solve. Two people can still book the same seat. Here's why: Alice and Bob can both start their transactions simultaneously, both check that `available_seats >= 1` (both see 1 seat available), and both execute their `UPDATE` statements. Since each transaction is atomic, both succeed, but now we've sold 2 tickets for 1 seat.

The issue is that transactions provide atomicity within themselves, but don't prevent other transactions from reading the same data concurrently. We need coordination mechanisms to solve this.

Pessimistic Locking

Pessimistic locking prevents conflicts by acquiring locks upfront. The name comes from being "pessimistic" about conflicts - assuming they will happen and preventing them.

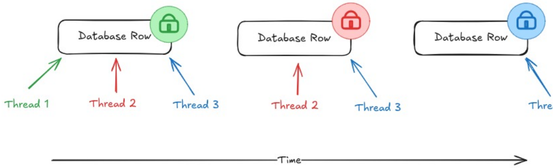
We can fix our race condition using explicit row locks:

> Of course, in a real ticketing system you'd have the concept of ticket reservations to improve the user experience. But for the sake of this example, we'll keep it simple and we talk about how to handle reservations in a later section.

```sql
BEGIN TRANSACTION;

-- Lock the row first to prevent race conditions
SELECT available_seats FROM concerts
WHERE concert_id = 'weeknd_tour'
FOR UPDATE;

-- Now safely update the seat count
UPDATE concerts
SET available_seats = available_seats - 1
WHERE concert_id = 'weeknd_tour'

-- Create the ticket record
INSERT INTO tickets (user_id, concert_id, seat_number, purchase_time)
VALUES ('user123', 'weeknd_tour', 'A15', NOW());

COMMIT;
```

The `FOR UPDATE` clause acquires an exclusive lock on the concert row before reading. When Alice runs this code, Bob's identical transaction will block at the SELECT statement until Alice's transaction completes. This prevents both from seeing the same initial seat count and ensures only one person can check and update at a time.

A **lock** in this context is a mechanism that prevents other database connections from accessing the same data until the lock is released. Databases like PostgreSQL and MySQL are multi-threaded systems that can handle thousands of concurrent connections, but locks ensure that only one connection can modify a specific row (or set of rows) at a time.



Explicit Row Locks

Performance considerations are really important when using locks. You want to lock as few rows as possible for as short a time as possible. Lock entire tables and you kill concurrency. Hold locks for seconds instead of milliseconds and you create bottlenecks. In our example, we're only locking one specific concert row briefly during the purchase.

Isolation Levels

Instead of explicitly locking rows with `FOR UPDATE`, you can let the database automatically handle conflicts by raising what's called the isolation level. **Isolation levels** control how much concurrent transactions can see of each other's changes. Think of it as how "isolated" each transaction is from seeing other transactions' work.

Most databases support four standard isolation levels (these are different options, not a progression):

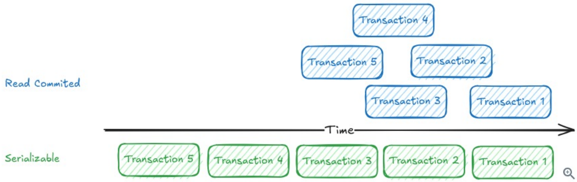
- `READ UNCOMMITTED` - Can see uncommitted changes from other transactions (rarely used)
- `READ COMMITTED` - Can only see committed changes (default in PostgreSQL)
- `REPEATABLE READ` - Same data read multiple times within a transaction stays consistent (default in MySQL)
- `SERIALIZABLE` - Strongest isolation, transactions appear to run one after another

The defaults of either `READ COMMITTED` or `REPEATABLE READ` still allows our concert ticket race condition because both Alice and Bob can read "1 seat available" simultaneously before updating. The `SERIALIZABLE` isolation level solves this by making transactions appear to run one at a time:

```sql
-- Set isolation level for this transaction
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

UPDATE concerts
```

```
SET available_seats = available_seats - 1
WHERE concert_id = 'weeknd_tour'

-- Create the ticket record
INSERT INTO tickets (user_id, concert_id, seat_number, purchase_time)
VALUES ('user123', 'weeknd_tour', 'A15', NOW());

COMMIT;
```

With `SERIALIZABLE`, the database automatically detects conflicts and aborts one transaction if they would interfere with each other. The aborted transaction must retry.



Isolation Levels

The tradeoff is that `SERIALIZABLE` isolation is much more expensive than explicit locks. It requires the database to track all reads and writes to detect potential conflicts, and transaction aborts waste work that must be redone. Explicit locks give you precise control over what gets locked and when, making them more efficient for scenarios where you know exactly which resources need coordination.

Optimistic Concurrency Control

Pessimistic locking assumes conflicts will happen and prevents them upfront. **Optimistic concurrency control (OCC)** takes the opposite approach in that it assumes conflicts are rare and detects them after they occur.

The performance benefit is significant. Instead of blocking transactions waiting for locks, you let them all proceed and only retry the ones that conflict. Under low contention, this eliminates locking overhead entirely.

The pattern is simple, you can include a version number with your data. Every time you update a record, increment the version. When updating, specify both the new value and the expected current version.

```
-- Alice reads: concert has 1 seat, version 42
-- Bob reads: concert has 1 seat, version 42

-- Alice tries to update first:
BEGIN TRANSACTION;
UPDATE concerts
SET available_seats = available_seats - 1, version = version + 1
WHERE concert_id = 'weeknd_tour'
  AND version = 42;  -- Expected version

INSERT INTO tickets (user_id, concert_id, seat_number, purchase_time)
VALUES ('alice', 'weeknd_tour', 'A15', NOW());
COMMIT;

-- Alice's update succeeds, seats = 0, version = 43

-- Bob tries to update:
BEGIN TRANSACTION;
UPDATE concerts
SET available_seats = available_seats - 1, version = version + 1
WHERE concert_id = 'weeknd_tour'
  AND version = 42;  -- Stale version!

-- Bob's update affects 0 rows - conflict detected, transaction rolls back
```

When Bob's update fails, he knows someone else modified the record. He can re-read the current state, check if seats are still available, and retry with the new version number. If seats are gone, he gets a clear "sold out" message instead of a mysterious failure.

Importantly, the "version" doesn't have to be a separate column. You can use existing data that naturally changes when the record is updated. In our concert example, the available seats count itself serves as the version. Here's how it works:

```
-- Alice reads: 1 seat available
-- Bob reads: 1 seat available

-- Alice tries to update first:
BEGIN TRANSACTION;
UPDATE concerts
SET available_seats = available_seats - 1
WHERE concert_id = 'weeknd_tour'
  AND available_seats = 1;  -- Expected current value

INSERT INTO tickets (user_id, concert_id, seat_number, purchase_time)
VALUES ('alice', 'weeknd_tour', 'A15', NOW());
COMMIT;

-- Alice's update succeeds, seats now = 0

-- Bob tries to update:
BEGIN TRANSACTION;
UPDATE concerts
SET available_seats = available_seats - 1
WHERE concert_id = 'weeknd_tour'
  AND available_seats = 1;  -- Stale value!

-- Bob's update affects 0 rows - conflict detected, transaction rolls back
```

This approach works because we're checking that the current seat count matches what we read earlier. If someone else bought the ticket first, the seat count changed and our update fails.

The same pattern applies to other scenarios. For eBay bidding, use the current highest bid amount as the version. For bank transfers, use the account balance. For inventory systems, use the stock count. Any value that changes when the record is updated can serve as your optimistic concurrency control mechanism.

> You'll need to be careful to avoid what is called the ABA problem. Where thread A reads a value (A), thread B changes it to B then back to A, and thread A thinks nothing changed when it does its compare-and-swap. This can happen with optimistic locking when using simple version numbers that wrap around, or when memory gets reused. More on this when we get into Common Deep Dives.

This approach makes sense when conflicts are uncommon. For most e-commerce scenarios, the chance of two people buying the exact same item at the exact same moment is low. The occasional retry is worth avoiding the overhead of pessimistic locking.

Multiple Nodes

All the approaches we've covered so far work within a single database. But what happens when you need to coordinate updates across multiple databases? This is where things get significantly more complex.

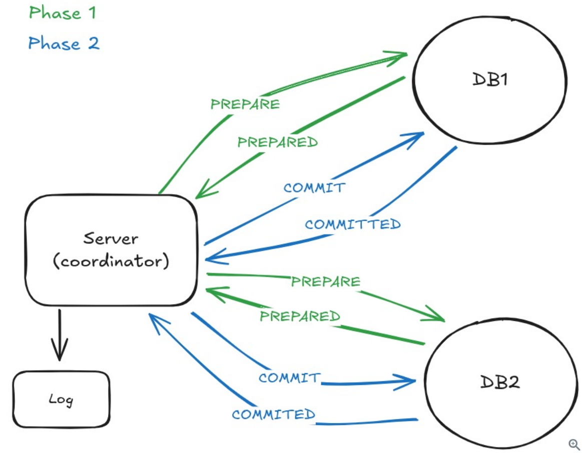
> If you identify that your system needs strong consistency guarantees during high-contention scenarios, you should do all you can to keep the relevant data in a single database. Nine times out of ten, this is entirely possible and avoids the need for distributed coordination, which can get ugly fast.

Consider a bank transfer where Alice and Bob have accounts in different databases. Maybe your bank grew large enough that you had to shard user accounts across multiple databases. Alice's account lives in Database A while Bob's account lives in Database B. Now you can't use a single database transaction to handle the transfer. Database A needs to debit $100 from Alice's account while Database B needs to credit $100 to Bob's account. Both operations must succeed or both must fail. If Database A debits Alice but Database B fails to credit Bob, money disappears from the system.

You have several options for distributed coordination, each with different trade-offs:

Two-Phase Commit (2PC)

The classic solution is two-phase commit, where your transfer service acts as the coordinator managing the transaction across multiple database participants. The coordinator (your service) asks all participants to prepare the transaction in the first phase, then tells them to commit or abort in the second phase based on whether everyone successfully prepared.



Two-Phase Commit

Critically, the coordinator must write to a persistent log before sending any commit or abort decisions. This log records which participants are involved and the current state of the transaction. Without this log, coordinator crashes create unrecoverable situations where participants don't know whether to commit or abort their prepared transactions.

> Keeping transactions open across network calls is extremely dangerous. Those open transactions hold locks on Alice's and Bob's account rows, blocking any other operations on those accounts. If your coordinator service crashes, those transactions stay open indefinitely, potentially locking the accounts forever. Production systems add timeouts to automatically rollback prepared transactions after 30-60 seconds, but this creates other problems like legitimate slow operations might get rolled back, causing the transfer to fail even when it should have succeeded.

The prepare phase is where each database does all the work except the final commit. Database A starts a transaction, verifies Alice has sufficient funds, places a hold on $100, but doesn't commit yet. The changes are made but not permanent, and other transactions can't see them. Database B starts a transaction, verifies Bob's account exists, prepares to add $100, but doesn't commit yet. In SQL terms, this looks like:

```sql
-- Database A during prepare phase
BEGIN TRANSACTION;
SELECT balance FROM accounts WHERE user_id = 'alice' FOR UPDATE;
-- Check if balance >= 100
UPDATE accounts SET balance = balance - 100 WHERE user_id = 'alice';
-- Transaction stays open, waiting for coordinator's decision

-- Database B during prepare phase
BEGIN TRANSACTION;
SELECT * FROM accounts WHERE user_id = 'bob' FOR UPDATE;
-- Verify account exists and is active
UPDATE accounts SET balance = balance + 100 WHERE user_id = 'bob';
-- Transaction stays open, waiting for coordinator's decision
```

If both databases can prepare successfully, your service tells them to commit their open transactions. If either fails, both roll back their open transactions.

Two-phase commit guarantees atomicity across multiple systems, but it's expensive and fragile. If your service crashes between prepare and commit, both databases are left with open transactions in an uncertain state. If any database is slow or unavailable, the entire transfer blocks. Network partitions can leave the system in an inconsistent state.

Distributed Locks

For simpler coordination needs, you can use distributed locks. Instead of coordinating complex transactions, you just ensure only one process can work on a particular resource at a time across your entire system.

For our bank transfer, you could acquire locks on both Alice's and Bob's account IDs before starting any operations. This prevents concurrent transfers from interfering with each other:

Distributed locks can be implemented with several technologies, each with different characteristics:

Redis with TTL - Redis provides atomic operations with automatic expiration, making it ideal for distributed locks. The `SET` command with expiration atomically creates a lock that Redis will automatically remove after the TTL expires. This eliminates the need for cleanup jobs since Redis handles expiration in the background. The lock is distributed because all your application servers can access the same Redis instance and see consistent state. When the lock expires or is explicitly deleted, the resource becomes available again. The advantage is speed and simplicity. Redis operations are very fast and the TTL handles cleanup automatically. The disadvantage is that Redis becomes a single point of failure, and you need to handle scenarios where Redis is unavailable.

Database columns - You can implement distributed locks using your existing database by adding status and expiration columns to track which resources are locked. This approach keeps everything in one place and leverages your database's ACID properties to ensure atomicity when acquiring locks. A background job periodically cleans up expired locks, though you need to handle race conditions between the cleanup job and users trying to extend their locks. The advantage is consistency with your existing data and no additional infrastructure. The disadvantage is that database operations are slower than cache operations, and you need to implement and maintain cleanup logic.

ZooKeeper/etcd - These are purpose-built coordination services designed specifically for distributed systems. They provide strong consistency guarantees even during network partitions and leader failures. ZooKeeper uses ephemeral nodes that automatically disappear when the client session ends, providing natural cleanup for crashed processes. Both systems use consensus algorithms (Raft for etcd, ZAB for ZooKeeper) to maintain consistency across multiple nodes.

The advantage is robustness. These systems are designed to handle the complex failure scenarios that Redis and database approaches struggle with. The disadvantage is operational complexity, as you need to run and maintain a separate coordination cluster.

Distributed locks aren't just for technical coordination either, they can dramatically improve user experience by preventing contention before it happens. Instead of letting users compete for the same resource, create intermediate states that give temporary exclusive access.

Consider Ticketmaster seat reservations. When you select a seat, it doesn't immediately go from "available" to "sold." Instead, it goes to a "reserved" state that gives you time to complete payment while preventing others from selecting the same seat. The contention window shrinks from the entire purchase process (5 minutes) to just the reservation step (milliseconds).

The same pattern appears everywhere. Uber sets driver status to "pending_request," e-commerce sites put items "on hold" in shopping carts, and meeting room booking systems create temporary holds.

The advantage is simplicity compared to complex transaction coordination. The disadvantage is that distributed locks can become bottlenecks under high contention, and you need to handle lock timeouts and failure scenarios.

Saga Pattern

The saga pattern takes a different approach. Instead of trying to coordinate everything atomically like 2PC, it breaks the operation into a sequence of independent steps that can each be undone if something goes wrong.

Think of it like this. Instead of holding both Alice's and Bob's accounts locked while coordinating, you just do the operations one by one. First, debit Alice's account and commit that transaction immediately. Then, credit Bob's account and commit that transaction. If the second step fails, you "compensate" by crediting Alice's account back to undo the first step.

For our bank transfer example

1. **Step 1** - Debit $100 from Alice's account in Database A, commit immediately
2. **Step 2** - Credit $100 to Bob's account in Database B, commit immediately
3. **Step 3** - Send confirmation notifications

If Step 2 fails (Bob's account doesn't exist), you run the compensation for Step 1. You credit $100 back to Alice's account. If Step 3 fails, you compensate both Step 2 (debit Bob's account) and Step 1 (credit Alice's account).

Each step is a complete, committed transaction. There are no long-running open transactions and no coordinator crashes leaving things in limbo. Each database operation succeeds or fails independently.

But there is (of course) an important tradeoff. During saga execution, the system is temporarily inconsistent. After Step 1 completes, Alice's account is debited but Bob's account isn't credited yet. Other processes might see Alice's balance as $100 lower during this window. If someone checks the total money in the system, it appears to have decreased temporarily.

This eventual consistency is what makes sagas practical. You avoid the fragility of 2PC by accepting that the system will be briefly inconsistent. You handle this by designing your application to understand these intermediate states. For example, you might show transfers as "pending" until all steps complete.

Choosing the Right Approach

> Keep in mind, like with much of system design, there isn't always a clear-cut answer. You'll need to consider the tradeoffs of each approach based on your specific use case and make the appropriate justification for your choice.

Start here. Can you keep all the contended data in a single database? If yes, use pessimistic locking or optimistic concurrency based on your conflict frequency.

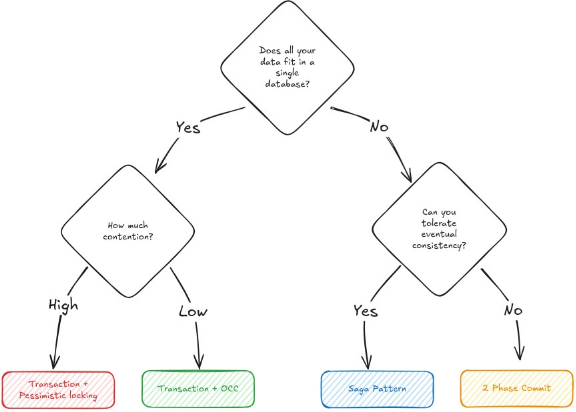
Single database, high contention: Pessimistic locking with explicit locks (`FOR UPDATE`). This provides predictable performance, is simple to reason about, and handles worst-case scenarios well.

Single database, low contention: Optimistic concurrency control using existing columns as versions. This provides better performance when conflicts are rare and has no blocking.

Multiple databases, must be atomic: Distributed transactions (2PC for strong consistency, Sagas for resilience). Use only when you absolutely need atomicity across systems.

User experience matters: Distributed locks with reservations to prevent users from entering contention scenarios. This is great for ticketing, e-commerce, and any user-facing competitive flows.

Approach	Use When	Avoid When	Typical Latency	Complexity
Pessimistic Locking	High contention, critical consistency, single database	Low contention, high throughput needs	Low (single DB query)	Low
SERIALIZABLE Isolation	Need automatic conflict detection, can't identify specific locks	Performance critical, high contention	Medium (conflict detection overhead)	Low
Optimistic Concurrency	Low contention, high read/write ratio, performance critical	High contention, can't tolerate retries	Low (when no conflicts)	Medium
Distributed Transactions	Must have atomicity across systems, can tolerate complexity	High availability requirements, performance critical	High (network coordination)	Very High
Distributed Locks	User-facing flows, need reservations, simpler than 2PC	No alternatives available, purely technical coordination	Low (simple status updates)	Medium


Does all your data fit in a single database?
Yes
No
How much contention?
Can you tolerate eventual consistency?
High
Low
Yes
No
Transaction + Pessimistic locking
Transaction + OCC
Saga Pattern
2 Phase Commit

Flow Chart

> When in doubt, start with pessimistic locking in a single database. It's simple, predictable, and you can always improve it later.

When to Use in Interviews

Don't wait for the interviewer to ask about contention. Be proactive in recognizing scenarios where multiple processes might compete for the same resource and suggest appropriate coordination mechanisms. This is typically when you determine during your non-functional requirements that your system requires strong consistency.

Recognition Signals

Here are some bang on examples of when you might need to use contention patterns:

Multiple users competing for limited resources such as concert tickets, auction bidding, flash sale inventory, or matching drivers with riders

Prevent double-booking or double-charging in scenarios like payment processing, seat reservations, or meeting room scheduling

Ensure data consistency under high concurrency for operations like account balance updates, inventory management, or collaborative editing

Handle race conditions in distributed systems in any scenario where the same operation might happen simultaneously across multiple servers and where the outcome is sensitive to the order of operations.

Common Interview Scenarios

This shows up A LOT in common interview questions. It's one of the most popular patterns and interviewers love to ask about it. Here are some examples of places where you might need to use contention patterns:

Online Auction Systems - Perfect for demonstrating optimistic concurrency control because multiple bidders compete for the same item. You can use the current high bid as the "version" and only accept new bids if they're higher than the expected current bid. Application-level status coordination also helps by marking items as "bidding ends in 30 seconds" to prevent last-second contention scenarios.

Ticketmaster/Event Booking - While this seems like a classic pessimistic locking scenario for seat selection, application-level status coordination is actually the bigger win. When users select seats, you immediately reserve them with a 10-minute expiration, which prevents the terrible UX of users filling out payment info only to find the seat was taken by someone else.

Banking/Payment Systems - Great place to showcase distributed transactions since account transfers between different banks or services need atomic operations across multiple systems. You should start with saga pattern for resilience and mention 2PC only if the interviewer pushes for strict consistency requirements.

Ride Sharing Dispatch - Application-level status coordination shines here because you can set driver status to "pending_request" when sending ride requests, which prevents multiple simultaneous requests to the same driver. You can use either caches with TTL for automatic cleanup when drivers don't respond within 10 seconds, or database status fields with periodic cleanup jobs.

Flash Sale/Inventory Systems - Perfect for demonstrating a mix of approaches. You can use optimistic concurrency for inventory updates with the current stock count as your version, but you should also implement application-level coordination for shopping cart "holds" to improve user experience and reduce contention at checkout.

Yelp/Review Systems - Great example of optimistic concurrency control because when users submit reviews, you need to update the business's average rating. Multiple concurrent reviews for the same restaurant create contention, so you can use the current rating and review count as your "version" and only update if they match what you read initially. This prevents rating calculations from getting corrupted when reviews arrive simultaneously.

The best candidates identify contention problems before they're asked. When designing any system with shared resources, immediately address coordination:

"This auction system will have multiple bidders competing for items, so I'll use optimistic concurrency control with the current high bid as my version check."

"For the ticketing system, I want to avoid users losing seats after filling out payment info, so I'll implement seat reservations with a 10-minute timeout."

"Since we're sharding user accounts across databases, transfers between different shards will need distributed transactions. I'll use the saga pattern for resilience."

When NOT to overcomplicate

Don't reach for complex coordination mechanisms when simpler solutions work.

> ⚠ A common mistake I see is candidates reaching for distributed locks (Redis, etc) when a simple database transaction with row locking or OCC is sufficient. Keep in mind that adding new components adds system complexity and introduces new failure modes so do what you can to avoid them.

Low contention scenarios where conflicts are rare (like updating product descriptions where only admins can edit) can use basic optimistic concurrency with retry logic. Don't implement elaborate locking schemes when simple retry logic handles the occasional conflict.

Single-user operations like personal todo lists, private documents, or user preferences have no contention, so no coordination is needed.

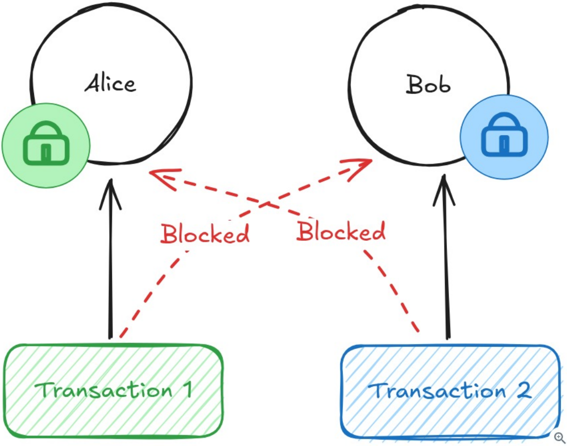
Read-heavy workloads where most operations are reads with occasional writes can use simple optimistic concurrency to handle the rare write conflicts without impacting read performance.

Common Deep Dives

Interviewers love to probe your understanding of edge cases and failure scenarios. Here are the most common follow-up questions when discussing contention patterns that end up being most common.

"How do you prevent deadlocks with pessimistic locking?"

Consider a bank transfer between two accounts. Alice wants to transfer $100 to Bob, while Bob simultaneously wants to transfer $50 to Alice. Transaction A needs to debit Alice's account and credit Bob's account. Transaction B needs to debit Bob's account and credit Alice's account. Transaction A locks Alice's account first, then tries to lock Bob's account. Transaction B locks Bob's account first, then tries to lock Alice's account. Both transactions wait forever for the other to release their lock.



Deadlock

This deadlock happens because the transactions acquire locks in different orders. The business logic doesn't care about order, it just wants to update both users when they interact. But databases can't read your mind about which locks are safe to acquire simultaneously.

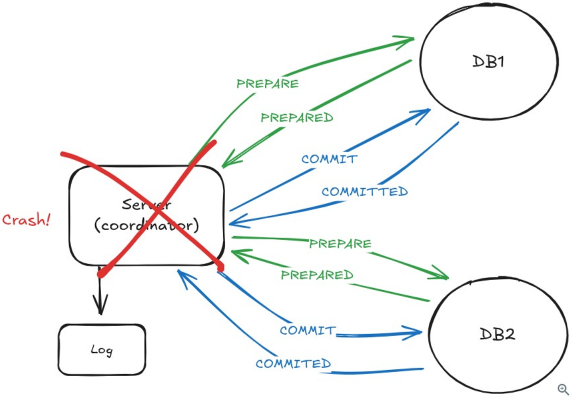
The standard solution is ordered locking, which means always acquiring locks in a consistent order regardless of your business logic flow. Sort the resources you need to lock by some deterministic key like user ID, database primary key, or even memory address. If you need to lock users 123 and 456, always lock 123 first even if your business logic processes 456 first. This prevents circular waiting because all transactions follow the same acquisition order.

In practice, this might mean sorting your user IDs before locking them, or establishing conventions like "always lock the user initiating the action first, then lock recipients in ascending ID order." The exact ordering doesn't matter as long as it's consistent across all transactions in your system.

As a fallback, database timeout configurations serve as your safety net when ordered locking isn't practical or when you miss edge cases. Set transaction timeouts so deadlocked transactions get killed after a reasonable wait period and can retry with proper ordering. Most modern databases also have automatic deadlock detection that kills one transaction when cycles are detected, but this should be your fallback, not your primary strategy.

"What if your coordinator service crashes during a distributed transaction?"

This is the classic 2PC failure scenario. Databases are sitting with prepared transactions, waiting for commit or abort instructions that never come. Those transactions hold locks on resources, potentially blocking other operations indefinitely.



2PC Failure

Production systems handle this with coordinator failover and transaction recovery. When a new coordinator starts up, it reads persistent logs to determine which transactions were in-flight and completes them. Most enterprise transaction managers (like Java's JTA) handle this automatically, but you still need to design for coordinator high availability and maintain transaction state across failures.

Sagas are more resilient here (as discussed earlier) because they don't hold locks across network calls. Coordinator failure just pauses progress rather than leaving participants in limbo.

"How do you handle the ABA problem with optimistic concurrency?"

Sneaky question that tests deeper understanding. The ABA problem occurs when a value changes from A to B and back to A between your read and write. Your optimistic check sees the same value and assumes nothing changed, but important state transitions happened.

Consider a review system like Yelp, where users can review businesses and each business tracks an average rating so we don't need to recalculate it each time. A restaurant starts with 4.0 stars and 100 reviews. Two new reviews come in simultaneously - one gives 5 stars, another gives 3 stars. Both reviews see the current average as 4.0 and calculate the new average. Due to the math, the final average might still end up at 4.0 stars, but now with 102 reviews. If you use just the average rating as your "version," both updates would succeed because they see the same 4.0 value, but you'd miss one of the reviews.

The solution is using a column that you know will always change. In the Yelp case, use the review count instead of the average rating as your optimistic concurrency check. Every new review increases the count, so it's a perfect monotonically increasing version. Your update becomes "set new average and increment count to 101, but only if current count is 100."

```sql
-- Use review count as the "version" since it always increases
UPDATE restaurants
SET avg_rating = 4.1, review_count = review_count + 1
WHERE restaurant_id = 'pizza_palace'
  AND review_count = 100;  -- Expected current count
```

When you can't find a naturally changing column, fall back to an explicit version column that increments on every update regardless of whether your business data actually changed. Even if the average rating stays the same, the version number shows that processing occurred.

Some databases solve this automatically. DynamoDB's conditional writes include an internal version that increments on every update, not just when your data changes.

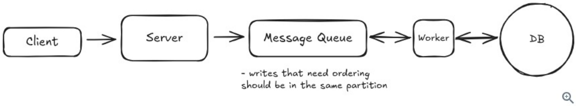
"What about performance when everyone wants the same

resource?"

This is the hot partition or celebrity problem, where your carefully designed distributed system suddenly has everyone hammering the same single resource. Think about what happens when a celebrity joins Twitter and millions of users try to follow them simultaneously, or when a rare collectible drops on eBay and thousands of people bid on the same item, or when Taylor Swift announces a surprise concert and everyone tries to buy tickets at the exact same time.

The fundamental issue is that normal scaling strategies break down when demand concentrates on a single point. Sharding doesn't help because you can't split one Taylor Swift concert across multiple databases because everyone wants that specific resource. Load balancing doesn't help because all the load balancer does is distribute requests to different servers that then compete for the same database row. Even read replicas don't help because the bottleneck is on the writes.

Your first strategy should be questioning whether you can change the problem itself rather than throwing more infrastructure at it. Maybe instead of one auction item, you actually have 10 identical items and can run separate auctions for each. Maybe instead of requiring immediate consistency for social media interactions, you can make likes and follows eventually consistent - users won't notice if their follow takes a few seconds to appear on the celebrity's follower count.



Queue-Based Serialization

For cases where you truly need strong consistency on a hot resource, implement queue-based serialization. Put all requests for that specific resource into a dedicated queue that gets processed by a single worker thread. This eliminates contention entirely by making operations sequential rather than concurrent. The queue acts as a buffer that can absorb traffic spikes while the worker processes requests at a sustainable rate.

The tradeoff is latency. Users might wait longer for their requests to be processed. But this is often better than the alternative of having your entire system grind to a halt under the contention.

Conclusion

Contention handling is crucial for reliable systems, but the path to success isn't what most engineers think. You should exhaust every single-database solution before even considering distributed coordination, since modern databases like PostgreSQL can handle tens of terabytes and thousands of concurrent connections. This covers the vast majority of applications you'll ever build, and the complexity jump to distributed coordination comes with enormous overhead and often worse performance.

You should stay within a single database as long as possible because both pessimistic locking and optimistic concurrency give you simple, battle-tested solutions with ACID guarantees. Pessimistic locking handles high contention predictably, while optimistic concurrency delivers excellent performance when conflicts are rare. Only move to distributed coordination when you've truly outgrown vertical scaling or need geographic distribution, which happens much later than most engineers think.

The best system designers fight hard to keep their data together and choose the right coordination pattern for their specific consistency requirements. Master these fundamentals, but always remember that the simplest solution that works is almost always the right choice.

Test Your Knowledge

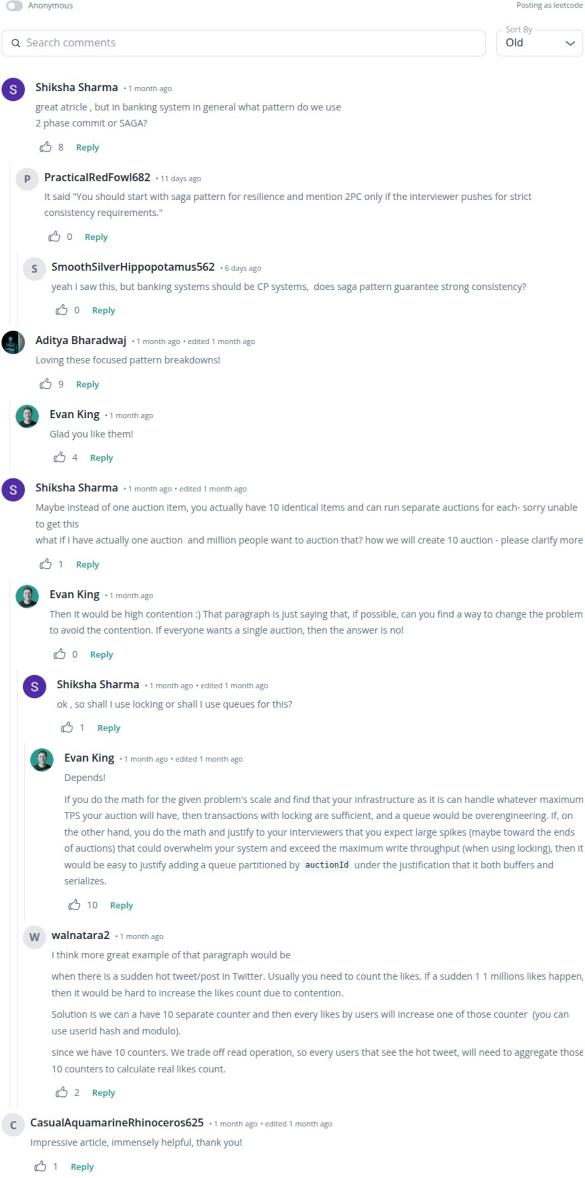
Take a quick 15 question quiz to test what you've learned.

Start Quiz

Mark as read

Add a comment...

Comment



Shiksha Sharma • 1 month ago

great atricle , but in banking system in general what pattern do we use
2 phase commit or SAGA?

8 Reply

PracticalRedFowl682 • 11 days ago

It said "You should start with saga pattern for resilience and mention 2PC only if the interviewer pushes for strict consistency requirements."

0 Reply

SmoothSilverHippopotamus562 • 6 days ago

yeah I saw this, but banking systems should be CP systems, does saga pattern guarantee strong consistency?

0 Reply

Aditya Bharadwaj • 1 month ago • edited 1 month ago

Loving these focused pattern breakdowns!

9 Reply

Evan King • 1 month ago

Glad you like them!

4 Reply

Shiksha Sharma • 1 month ago • edited 1 month ago

Maybe instead of one auction item, you actually have 10 identical items and can run separate auctions for each- sorry unable to get this
what if I have actually one auction and million people want to auction that? how we will create 10 auction - please clarify more

1 Reply

Evan King • 1 month ago

Then it would be high contention :) That paragraph is just saying that, if possible, can you find a way to change the problem to avoid the contention. If everyone wants a single auction, then the answer is no!

0 Reply

Shiksha Sharma • 1 month ago • edited 1 month ago

ok , so shall I use locking or shall I use queues for this?

1 Reply

Evan King • 1 month ago • edited 1 month ago

Depends!

If you do the math for the given problem's scale and find that your infrastructure as it is can handle whatever maximum TPS your auction will have, then transactions with locking are sufficient, and a queue would be overengineering. If, on the other hand, you do the math and justify to your interviewers that you expect large spikes (maybe toward the ends of auctions) that could overwhelm your system and exceed the maximum write throughput (when using locking), then it would be easy to justify adding a queue partitioned by `auctionId` under the justification that it both buffers and serializes.

10 Reply

walnatara2 • 1 month ago

I think more great example of that paragraph would be

when there is a sudden hot tweet/post in Twitter. Usually you need to count the likes. If a sudden 1 1 millions likes happen, then it would be hard to increase the likes count due to contention.

Solution is we can a have 10 separate counter and then every likes by users will increase one of those counter (you can use userId hash and modulo).

since we have 10 counters. We trade off read operation, so every users that see the hot tweet, will need to aggregate those 10 counters to calculate real likes count.

2 Reply

CasualAquamarineRhinoceros625 • 1 month ago • edited 1 month ago

Impressive article, immensely helpful, thank you!

1 Reply

C **CasualAquamarineRhinoceros625** • 1 month ago • edited 1 month ago

Under OCC you have "For most e-commerce scenarios, the chance of two people buying the exact same item at the exact same moment is low." - is this true at Amazon's scale and during events like Prime Day or in Flash sale type situations as well ?

👍 1 Reply

B **BottomTealSkunk679** • 28 days ago

For those situations, you can use a good and fast, but imperfect, lock in front of the DB. I think this is covered in the Ticketmaster question

👍 0 Reply

Show All Comments

Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

Schedule A Mock Interview

Questions

Meta SWE Interview Questions
Amazon SWE Interview Questions
Google SWE Interview Questions
Engineering Manager (EM) Interview Questions

Learn

Learn System Design
Learn DSA
Learn Behavioral
Learn ML System Design
Guided Practice

Links

FAQ
Pricing
Gift Mock Interviews
Gift Premium
Become a Coach
Our Coaches
Employers
Hello Interview Premium

Legal

Terms and Conditions
Privacy Policy

Contact

About Us
support@hellointerview.com
7511 Greenwood Ave North
Unit #4238 Seattle
WA 98103

© 2025 Optick Labs Inc. All rights reserved.